MECHATRONIC SYSTEM INTEGRATION
(MCTA 3203)

SEMESTER 1, 2026/2027

LAB REPORT 3

MCTA 3203

SECTION 1

GROUP 8

NO.	MATRIC NO.	NAME
1	2318483	SYED MOHAMAD SYAFIEE BIN SYED TAJUDIN
2	2413869	SHARIF NUR AZWAR BIN SHARIF ABU BAKAR
3	2413733	ZAHIN ZAYDAN BIN ZAIRY

SUBMISSION DATE : 25 OCTOBER 2026

Abstract

This experiment involves the development of a motion-activated RFID access control system using an Arduino, MPU6050 sensor, RFID module, and servo motor. In Task 1, the MPU6050 sensor was used to detect and monitor hand movements, with real-time motion data displayed in PyCharm using Python. In Task 2, the MPU6050 sensor and RFID module were integrated to create an access control system that combines motion detection with RFID authentication. Through this experiment, we gained a better understanding of sensor integration, serial communication between Arduino and Python, and the application of motion detection and RFID technology in automated security systems.The MPU6050 module is a powerful MEMS sensor combining a 3-axis accelerometer and gyroscope to track motion, tilt, and orientation with remarkable precision. Featuring an integrated Digital Motion Processor (DMP) and a 16-bit ADC, it handles complex calculations on-chip, significantly easing the processing workload on microcontrollers like Arduino. Ideal for applications ranging from racing drones and self-balancing robots to interactive wearable devices, its I2C communication protocol ensures simple wiring and seamless integration. Ultimately, the MPU6050 transforms static hardware into dynamic, motion-aware projects, serving as an indispensable gateway for makers bringing real-world physical interactions to life.

Table of Content

Abstract

1. Introduction

2. Material and Equipment

3. Experiment Setup

4. Methodology

5. Data Collection

6. Data Analysis

7. Result

8. Conclusion

9. Recommendations

10. References

11. Appendices

12. Acknowledgement

13. Student Declaration

WEEK 4: SERIAL COMMUNICATION BETWEEN MICROCONTROLLERS, SENSORS, AND ACTUATORS

1. Introduction

Nowadays, automation and security systems play an important role in improving safety and convenience. Applications such as smart doors, motion detectors, and keyless entry systems use sensors and microcontrollers to detect movement and manage access automatically. These technologies help make security systems more efficient and reliable.

In this laboratory experiment, a smart access control system was developed using two main components: the MPU6050 motion sensor and an RFID reader. The MPU6050 detects movement and orientation, while the RFID reader identifies users by scanning RFID cards or tags. By combining these two components, the system can integrate motion detection and RFID authentication to control access, similar to modern automated security systems. The pin configuration of the MPU6050 sensor is illustrated in **Figure 1.0**.

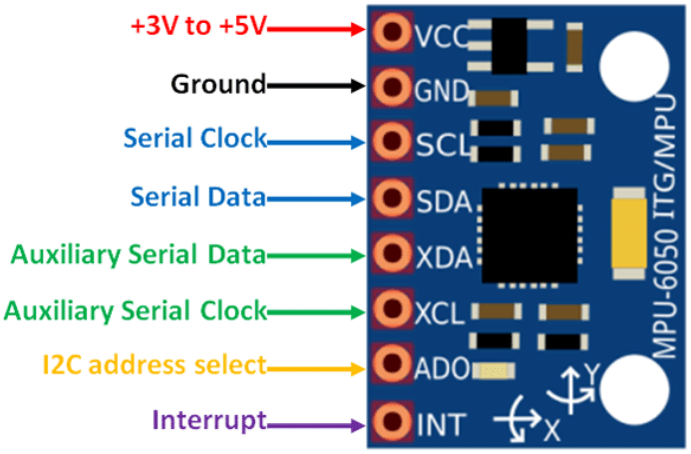


Figure 1.0 Shows the pin for MPU6050

Arduino was used to control the hardware components, including the servo motor, which functioned as a door lock, and the LEDs, which indicated the system status. Meanwhile, Python was used on the computer to receive and process real-time sensor data from the Arduino through serial communication. This enabled data exchange between the hardware and software components.

The main objectives of this experiment were:

- To interface the MPU6050 sensor and RFID module with the Arduino.
- To detect basic motion patterns using the MPU6050 sensor.
- To integrate motion detection and RFID authentication to control a servo-based access system.
- To establish serial communication between Python and Arduino for real-time data exchange.

2. Materials and Equipments

- Cytron CT-UNO 1
- MPU6050 sensor 1
- Light Emitting Diode, LED 2
- Servo Motor 1
- 220 Ω Resistor 2
- Breadboard 1
- RFID reader (MFRC522) + RFID tags/cards 1
- Computer with Arduino IDE & PyCharm 1
- Jumper wires

3. Experimental Setup

3.1 Task 1 - Motion Detection with MPU6050

In this task, we used the MPU6050 motion sensor to detect and monitor movement. The MPU6050 combines an accelerometer, which measures acceleration along three axes, and a gyroscope, which measures rotational movement and angular velocity. A total of five pins were used to establish the connections for this task.

The schematic diagram of the circuit connections is shown below:

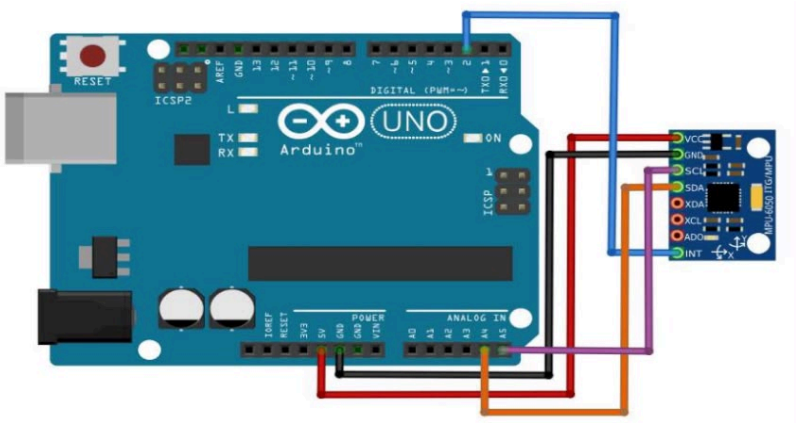


Figure 3.0 Shows the schematic diagram for Task 1

After connecting the sensor, we uploaded a program to the Arduino to continuously collect sensor readings and transmit them to the computer through serial communication. PyCharm was then used to run a Python program that received the data and displayed it as real-time graphs using the Matplotlib library. The graphs illustrated the changes in sensor readings as the MPU6050 was moved or tilted in different directions.

3.2 Task 2 - RFID + Servo Access System

In this task, the RFID reader, servo motor, LED indicators, and MPU6050 motion sensor were integrated to develop a basic access control system. The RFID reader was interfaced with the Arduino, while the servo motor was connected to pin D6. The green and red LEDs were connected to pins D5 and D7, respectively, with each LED using a 220 Ω current-limiting resistor.

The Arduino was programmed to read and verify RFID tags. When a valid tag was detected and the MPU6050 sensor registered the required motion, the green LED illuminated, and the

servo motor rotated to simulate unlocking a door. If an invalid RFID tag was scanned, the red LED turned on to indicate that access was denied. The MPU6050 sensor provided an additional motion-detection condition, allowing the system to combine RFID authentication and specific motion gestures for access control.

Schematic diagram of connection is shown below:

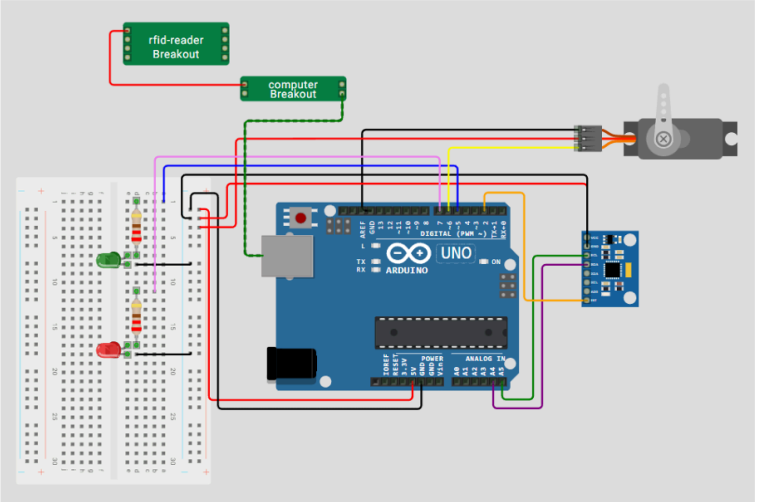


Figure 3.1 Shows the schematic diagram for Task 2

This experiment demonstrated how several electronic components can be integrated to operate together in a coordinated system. It also illustrated the role of the Arduino as the main controller, processing inputs from the RFID reader and MPU6050 sensor and controlling the output devices, including the servo motor and LED indicators, based on the detected conditions.

4. Methodology

4.1 Task 1

4.1.1 Circuit Assembly

The circuit was assembled according to the connections illustrated in Figure 3.0. The MPU6050 motion sensor was interfaced with the Arduino Uno, with its SDA pin connected to A4 and its SCL pin connected to A5. The sensor's power and ground pins were connected to the Arduino's 5V and GND pins, respectively.

The Arduino was powered through the computer's USB port, which supplied electrical power and enabled data communication. Jumper wires were used to establish the required circuit connections. This configuration enabled the Arduino to communicate with the MPU6050 sensor and collect sensor readings in real time.

4.1.2 Serial Communication Setup

The Arduino IDE was used to develop and upload a program that retrieved accelerometer and gyroscope readings from the MPU6050 sensor. The Arduino continuously collected the motion data and transmitted it to the computer through serial communication at a baud rate of 9600 bps.

On the computer, Python was executed using PyCharm to receive the incoming serial data. The program utilized the PySerial library to establish communication with the Arduino and obtain real-time sensor readings. Matplotlib was then used to plot the data dynamically, allowing the sensor's response to different movements, such as shaking and tilting, to be observed and analysed.

4.1.2.1 Code Upload to Arduino IDE

task1w4.ino

```ino
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

void setup() {
  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();
  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }
}

void loop() {
  int ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  Serial.print(ax); Serial.print(",");
  Serial.print(ay); Serial.print(",");
  Serial.println(az);
  delay(100);
}
```

4.1.3 Python Script

A Python script was developed in PyCharm to receive and display the motion data transmitted by the Arduino. The script utilized the PySerial library to read the incoming sensor readings and Matplotlib to plot the data in real time. When the MPU6050 sensor was shaken or moved in a circular direction, the graph showed corresponding spikes and oscillations. These changes demonstrated that the sensor could detect movement and reflect variations in motion through the live graph.

4.1.4. Matplotlib

The Arduino Serial Plotter was utilized to check the functionality of the MPU6050 sensor by displaying the accelerometer and gyroscope readings as continuously updating waveforms. The baud rate was set to 9600 to match the Arduino program, and the appropriate COM port was selected. When the sensor was moved or rotated, the plotted waveforms changed accordingly, indicating variations in acceleration and angular velocity. Before executing the Python script, the Serial Plotter was closed to prevent conflicts when accessing the same serial port.

4.1.4.1 Code Upload to PyCharm

```python
import serial
import matplotlib
matplotlib.use('TkAgg')  # Forces an interactive pop-up window in PyCharm
import matplotlib.pyplot as plt
import numpy as np
from collections import deque

# ================================================================
# IMPROVEMENT 1: Serial Configuration & Memory Buffer Limit
# Using deque(maxlen=35) to store only the 35 most recent data points.
# This prevents the plot from lagging and becoming cluttered over time.
# ================================================================
SERIAL_PORT = 'COM5'  # Change to match your Arduino COM port (e.g., 'COM4')
BAUD_RATE = 9600
MAX_POINTS = 35

ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
ser.flushInput()

x_vals = deque(maxlen=MAX_POINTS)
y_vals = deque(maxlen=MAX_POINTS)

```

```
22
23      # ================================================================
24      # IMPROVEMENT 2: Fixed Plot Syntax & Path Visualization
25      # Fixed the syntax error in ax.plot() from the starter code.
26      # Using 'ro-' (red dots connected by lines) to clearly visualize
27      # the continuous real-time hand motion path.
28      # ================================================================
29      plt.ion()
30      fig, ax = plt.subplots(figsize=(7, 7))
31      ax.set_xlim(-20000,  right: 20000)
32      ax.set_ylim(-20000,  top: 20000)
33      ax.set_title("Task 1: Real-Time MPU6050 Motion Tracking")
34      ax.set_xlabel("Acceleration X (ax)")
35      ax.set_ylabel("Acceleration Y (ay)")
36      ax.grid( visible: True, linestyle='--', alpha=0.6)
37
38      line, = ax.plot( *args: [], [], 'ro-', linewidth=2, markersize=5, label='Motion Path')
39      center_dot, = ax.plot( *args: [], [], 'bx', markersize=10, label='Circle Center')
40      status_box = ax.text(-18500,  y: 17500,  s: "Status: Collecting data...", fontsize=11, fontweight='bold',
41                           bbox=dict [str, str | float | int] (boxstyle='round', facecolor='wheat', alpha=0.8))
42      ax.legend(loc='lower right')
43
44
45      # ================================================================
```

```
44
45      # ================================================================
46      # IMPROVEMENT 3: Circular Motion Detection Algorithm
47      # Detects circular gestures based on 3 geometric criteria:
48      # 1. Motion amplitude (X and Y spans must exceed 6000 units).
49      # 2. Coverage across all 4 quadrants around the center point (centroid).
50      # 3. Radius consistency (standard deviation of radius relative to mean).
51      # ================================================================
52      def check_circular_gesture(x_arr :{__len__}, y_arr):  1 usage
53          if len(x_arr) < 16:
54              return False, 0, 0
55
56          x = np.array(x_arr)
57          y = np.array(y_arr)
58
59          # Calculate the center of motion (centroid)
60          cx, cy = np.mean(x), np.mean(y)
61
62          # 1. Check motion amplitude/span to ignore minor hand tremors or noise
63          span_x = np.max(x) - np.min(x)
64          span_y = np.max(y) - np.min(y)
65          if span_x < 6000 or span_y < 6000:
66              return False, cx, cy
```

```
    # 2. Verify that the path crosses all four quadrants around the center
    dx = x - cx
    dy = y - cy
    q1 = np.any((dx > 0) & (dy > 0))
    q2 = np.any((dx < 0) & (dy > 0))
    q3 = np.any((dx < 0) & (dy < 0))
    q4 = np.any((dx > 0) & (dy < 0))

    if not (q1 and q2 and q3 and q4):
        return False, cx, cy

    # 3. Evaluate circularity using radius consistency (standard deviation)
    radii = np.sqrt(dx ** 2 + dy ** 2)
    mean_radius = np.mean(radii)
    std_radius = np.std(radii)

    # If the relative standard deviation of the radius is < 0.45, it forms a circle
    is_circle = (std_radius / mean_radius) < 0.45
    return is_circle, cx, cy
```

```
print("Starting motion tracking... Move the sensor in a circular pattern. Press Ctrl+C to exit.")

while True:
    try:
        raw_line = ser.readline().decode('utf-8', errors='ignore').strip()
        if raw_line:
            values = raw_line.split(',')

            # ======================================================================
            # IMPROVEMENT 4: Fixed Serial Data Length Check
            # The Arduino sketch sends 3 values (ax, ay, az), not 6 as checked in
            # the starter code (len(values) == 6). Updated to len(values) == 3.
            # ======================================================================
            if len(values) == 3:
                ax_val = int(values[0])
                ay_val = int(values[1])

                x_vals.append(ax_val)
                y_vals.append(ay_val)
```

```
                # ================================================================
                # IMPROVEMENT 5: Moving Average Filter (Smoothing)
                # Smooths raw MPU6050 accelerometer readings to reduce sensor noise.
                # ================================================================
                if len(x_vals) >= 3:
                    smooth_x = np.convolve(x_vals, np.ones(3) / 3, mode='valid')
                    smooth_y = np.convolve(y_vals, np.ones(3) / 3, mode='valid')
                    line.set_data(smooth_x, smooth_y)
                else:
                    line.set_data(x_vals, y_vals)

                # Check if the current trajectory forms a circular motion
                detected, cx, cy = check_circular_gesture(x_vals, y_vals)
                if detected:
                    center_dot.set_data([cx], [cy])
                    status_box.set_text("Status: CIRCULAR MOTION DETECTED!")
                    status_box.set_bbox(dict [str, str | float | int] (boxstyle='round', facecolor='lightgreen', alpha=0.9)
                else:
                    center_dot.set_data([], [])
                    status_box.set_text("Status: Tracking motion...")
                    status_box.set_bbox(dict [str, str | float | int] (boxstyle='round', facecolor='wheat', alpha=0.8))
```

```
                plt.draw()
                plt.pause(0.02)

        except ValueError:
            # Ignore incomplete or corrupted serial lines during startup
            continue
        except KeyboardInterrupt:
            print("\nProgram terminated by user.")
            break

    ser.close()
    plt.close()
```

4.2 Task 2

4.2.1 Circuit Assembly

The integrated RFID and motion-based access control circuit was assembled using the same breadboard setup, as illustrated in Figure 3.1. The RFID module was interfaced with the Arduino Uno to enable tag detection and authentication.

The servo motor was connected to pin D6, while the red and green LED indicators were connected to pins D7 and D5, respectively, with each LED fitted with a 220 Ω current-limiting resistor. The MPU6050 sensor retained the same connections as in Task 1. The Arduino was powered through the computer's USB port, providing the necessary power and enabling serial communication.

4.2.2 Code Upload to Arduino IDE

The Arduino was responsible for managing both RFID authentication and motion verification. When an RFID tag was scanned, its unique identification number (UID) was retrieved and checked against the stored values. If the UID was valid, the system then waited for the required motion input from the MPU6050 sensor. Serial communication enabled data exchange between the Arduino and Python. Meanwhile, the Python script monitored the verification process and displayed the access status, indicating whether access was granted or denied based on the RFID authentication and motion detection results.

4.2.3. Code Upload to Arduino IDE

```
#include <Wire.h>
#include <MPU6050.h>
#include <Servo.h>

MPU6050 mpu;
Servo servo;
int servoPosition = 90; // Locked position

void setup() {
  Serial.begin(9600);

  // 1. Initialize Servo and LEDs first
  servo.attach(6);
  servo.write(servoPosition);
  pinMode(5, OUTPUT); // Green LED
  pinMode(7, OUTPUT); // Red LED

  // Default State: Locked (Red LED ON)
  digitalWrite(5, LOW);
  digitalWrite(7, HIGH);

  // 2. Initialize MPU6050
  Wire.begin();
  mpu.initialize();
  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }
}

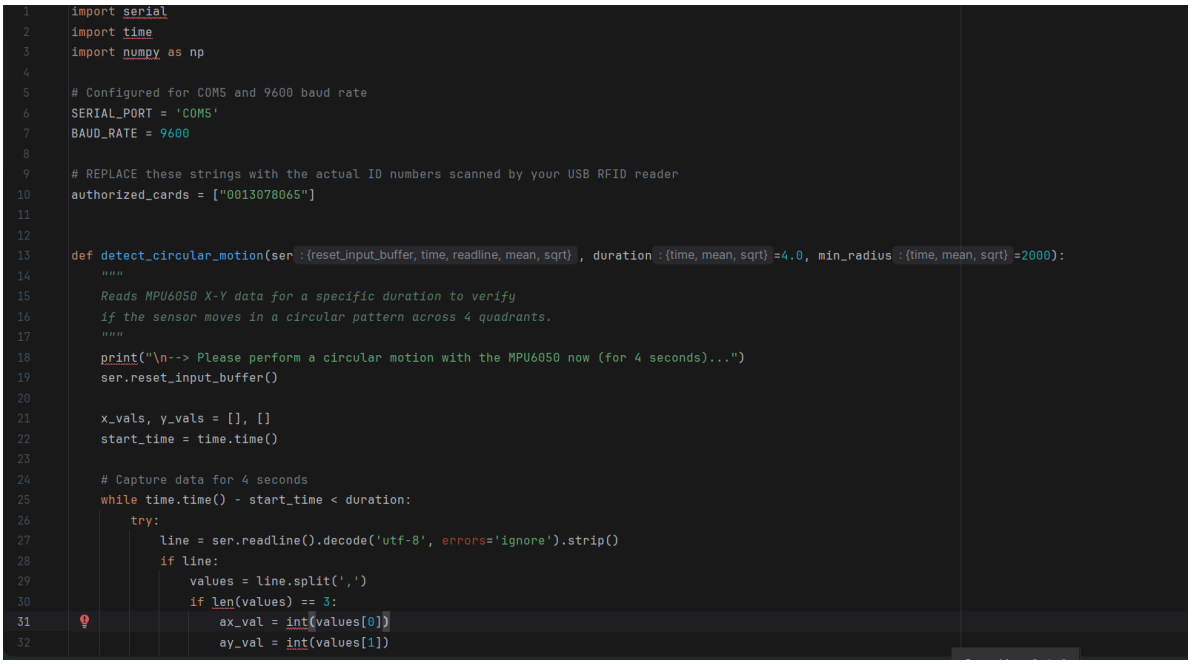
void loop() {
  // 1. Check for 'A' (Access Granted) or 'D' (Access Denied) from Python
  if (Serial.available() > 0) {
    char command = Serial.read();
    if (command == 'A') {
      servo.write(180);      // Unlock
      digitalWrite(5, HIGH); // Green LED ON
      digitalWrite(7, LOW);  // Red LED OFF
    } else if (command == 'D') {
      servo.write(90);       // Lock
      digitalWrite(5, LOW);  // Green LED OFF
      digitalWrite(7, HIGH); // Red LED ON
    }
  }

  // 2. Send MPU6050 accelerometer data to Python
  int16_t ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  Serial.print(ax); Serial.print(",");
  Serial.print(ay); Serial.print(",");
  Serial.println(az);

  delay(100);
}
```

4.2.3. Python Script for Servo and Visualization at PyCharm

A Python program was created in PyCharm to receive serial data from the Arduino and display feedback in text or visual form. The program monitored RFID scan results and motion verification status in real time. When access was approved, the Python terminal displayed a message confirming successful authentication. Conversely, when access was rejected, an error message appeared to indicate an invalid RFID card or an incorrect motion gesture.



```python
import serial
import time
import numpy as np

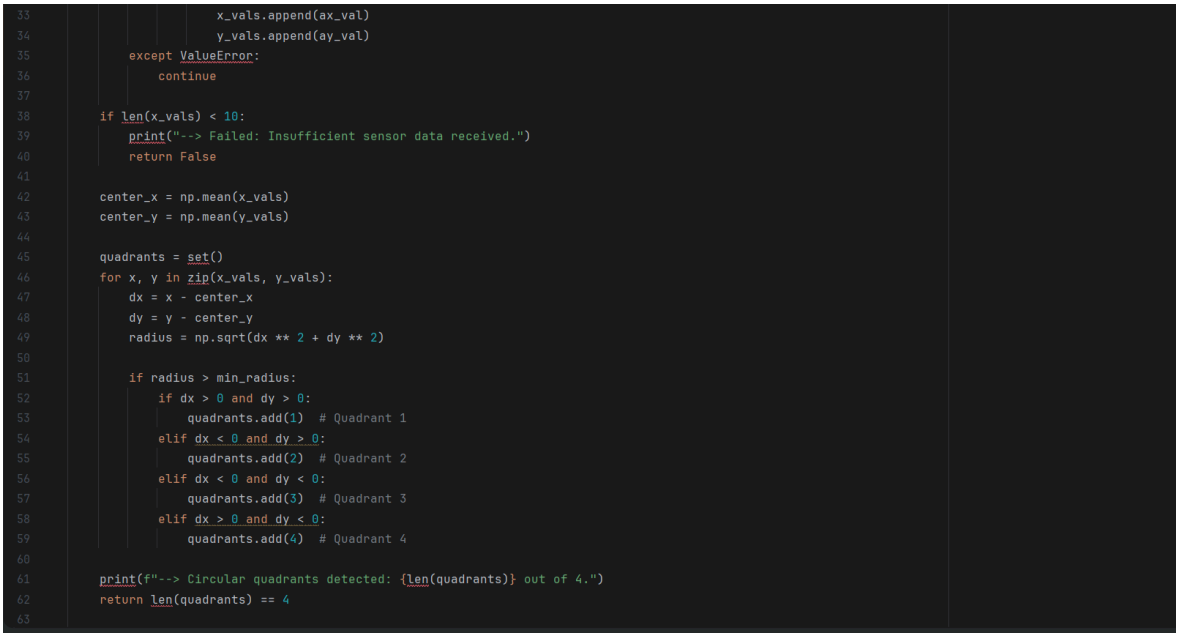
# Configured for COM5 and 9600 baud rate
SERIAL_PORT = 'COM5'
BAUD_RATE = 9600

# REPLACE these strings with the actual ID numbers scanned by your USB RFID reader
authorized_cards = ["0013078065"]


def detect_circular_motion(ser : {reset_input_buffer, time, readline, mean, sqrt} , duration : {time, mean, sqrt} =4.0, min_radius : {time, mean, sqrt} =2000):
    """
    Reads MPU6050 X-Y data for a specific duration to verify
    if the sensor moves in a circular pattern across 4 quadrants.
    """
    print("\n--> Please perform a circular motion with the MPU6050 now (for 4 seconds)...")
    ser.reset_input_buffer()

    x_vals, y_vals = [], []
    start_time = time.time()

    # Capture data for 4 seconds
    while time.time() - start_time < duration:
        try:
            line = ser.readline().decode('utf-8', errors='ignore').strip()
            if line:
                values = line.split(',')
                if len(values) == 3:
                    ax_val = int(values[0])
                    ay_val = int(values[1])
```



```python
                    x_vals.append(ax_val)
                    y_vals.append(ay_val)
        except ValueError:
            continue

    if len(x_vals) < 10:
        print("--> Failed: Insufficient sensor data received.")
        return False

    center_x = np.mean(x_vals)
    center_y = np.mean(y_vals)

    quadrants = set()
    for x, y in zip(x_vals, y_vals):
        dx = x - center_x
        dy = y - center_y
        radius = np.sqrt(dx ** 2 + dy ** 2)

        if radius > min_radius:
            if dx > 0 and dy > 0:
                quadrants.add(1)  # Quadrant 1
            elif dx < 0 and dy > 0:
                quadrants.add(2)  # Quadrant 2
            elif dx < 0 and dy < 0:
                quadrants.add(3)  # Quadrant 3
            elif dx > 0 and dy < 0:
                quadrants.add(4)  # Quadrant 4

    print(f"--> Circular quadrants detected: {len(quadrants)} out of 4.")
    return len(quadrants) == 4
```

```
    try:
        print(f"Connecting to Arduino on {SERIAL_PORT}...")
        ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
        time.sleep(2)
        print("=== SMART ACCESS SYSTEM READY ===")

        while True:
            # Click inside the terminal before scanning. The USB RFID types the ID here automatically.
            uid = input("\nClick here, then scan (tap) your RFID card: ").strip().upper()

            if not uid:
                continue

            print(f"Card Detected: '{uid}'")

            if uid in authorized_cards:
                print("RFID Status: AUTHORIZED. Proceed to motion verification.")
                if detect_circular_motion(ser):
                    print("Motion Status: VERIFIED! Access Granted (Servo Unlock, Green LED ON).")
                    ser.write(b'A')
                else:
                    print("Motion Status: FAILED! Access Denied (Servo Lock, Red LED ON).")
                    ser.write(b'D')
            else:
                print(f"RFID Status: DENIED! '{uid}' is not an authorized card.")
                ser.write(b'D')

    except serial.SerialException as e:
        print(f"\nFailed to open port {SERIAL_PORT}: {e}")
        print("Make sure the Serial Monitor in Arduino IDE is closed!")
    except KeyboardInterrupt:
        print("\nProgram terminated by user.")
    finally:
        if 'ser' in locals() and ser.is_open:
            ser.close()


if __name__ == "__main__":
    main()
```

5. Data Collection

The result is collected through observation by our own vision.

Task 1

1. When the MPU6050 motion sensor is moved either purposely or not, the graph plot shows a change of the plot's movement.
2. When a circular motion is made by the MPU6050, the plot will detect a circular movement and the center of the circular pattern will appear.

Task 2

1. When the RFID is tapped by a specific RFID card, a bleep will trigger, yellow LED will light up and the red LED will light off.
2. After the yellow LED is turned on, when a circular movement is made, the servo motor will move subsequently.
3. If there is no circular movement made,the red LED is turned on and the servo will not move.
4. However, when a wrong RFID card is tapped, the RFID will decline the authorization and the LED will remain red.

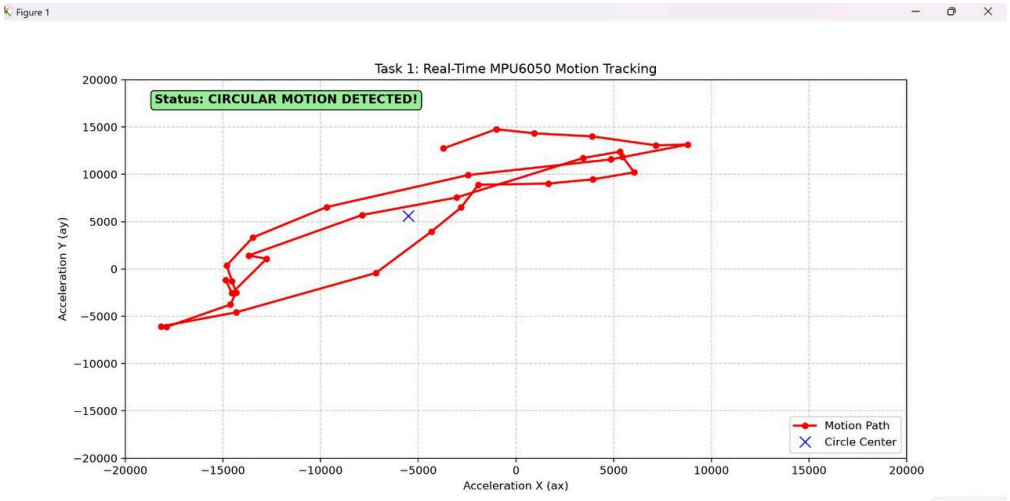
6. Data Analysis

The data confirmed that:

1. The RFID is working fine
2. The MPU6050 is working fine
3. The servo motor is working fine
4. The code is correctly written and working fine
5. The yellow and red LED is working fine

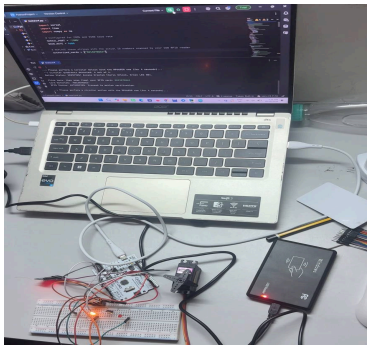
7. Results and Discussions

Task 1



The MPU6050 sensor successfully detected real-time hand movements, with the Python script plotting Acceleration X against Acceleration Y to visualize the continuous motion path. Upon detecting a valid circular gesture, the system marked the center of the pattern with an 'X' and updated the on-screen status to confirm the detection. Engineering-wise, the Arduino transmitted raw accelerometer data via serial communication at a 9600 baud rate to Python. The algorithm then verified the circular motion by ensuring the movement amplitude exceeded 6000 units, crossed all four quadrants around the centroid, and maintained a relative radius standard deviation below 0.45. To optimize performance, the system utilized a moving average filter to smooth sensor readings and restricted the memory buffer to 35 data points to prevent graphical lag. Potential sources of error in this setup include inherent sensor noise or hand tremors distorting the readings, incomplete or corrupted serial data lines during transmission, and human inconsistency in executing a precise circular motion that satisfies all strict geometric criteria.

Task 2



This activity imitates the action of paying the tolls on a highway. When the RFID

card is put on the scanner, the yellow LED will turn on indicating that the person who paid the toll is allowed to pass the toll, and the gate will be opened except for this experiment, in order for the gate to open (Servo Motor to move), a circular movement of MPU6050 must be made, and if only that the servo motor will move.

8.CONCLUSION

The experiment successfully demonstrated how to build an automated access control system by integrating an MPU6050 motion sensor and an RFID module with an Arduino. Establishing serial communication between the hardware and Python made it possible to track and analyze the sensor's acceleration data in real time. The setup effectively acted as a dual-step security check, where the servo motor would only unlock the simulated gate if a user scanned an authorized RFID card and performed a specific circular hand motion. Ultimately, this project provided valuable hands-on experience with connecting different electronic components, managing real-time data exchange, and applying these technologies to create modern smart security systems.

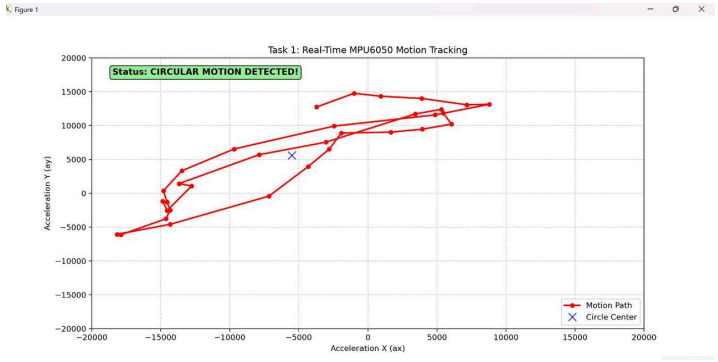
9.RECOMMENDATIONS

1. Use a Bluetooth or Wi-Fi microcontroller (like an ESP32) so the user can move the sensor freely without tangled jumper wires.
2. Solder the components instead of using a breadboard to prevent loose connections during hand movements
3. Include a small OLED or LCD screen for clear user instructions, rather than just relying on LEDs.

10.REFERENCE

Components101. (2021, March 17). *MPU6050 module pinout, configuration, features, Arduino interfacing & datasheet*. https://components101.com/sensors/mpu6050-module

11.APPENDICES



ACKNOWLEDGEMENT

Alhamdulillah, all praise and gratitude be to Allah SWT for His blessings, guidance, and mercy that enabled us to successfully complete this experiment.Throughout the development process, we faced several challenges, but with His will and guidance, we managed to overcome them and complete the project successfully. We would also like to express our heartfelt appreciation to our lecturer for their continuous guidance, support, and valuable feedback throughout this project. Their advice has greatly helped us in understanding both the theoretical and practical of digital logic systems and practical hardware interfacing using a Cytron CT-UNO microcontroller, MPU 6050 and Servo motor.

STUDENT DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons.We hereby certify that this report has not been done by only one individual and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate.We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.We therefore, agreed unanimously that this report shall be submitted for marking and this final The printed report has been verified by us.

SIGNATURE	STUDENT DETAILS	CHECKLIST
sharif	Name: Sharif Nur Azwar bin Sharif Abu Bakar Matric number : 2413869	Read [/] Understand [/] Agree [/]
zahin	Name: Zahin Zaydan bin Zairy Matric number :2413733	Read [/] Understand [/] Agree [/]
syed	Name: Syed Mohammad Syafiee bin Syed Tajudin Matric number : 2318483	Read [/] Understand [/] Agree [/]